

BO3: Workload-Specific Compilation Pipelines for Multi-Level Compilers

Megan Kuo , Atharva Khedkar , *Graduate Student Member, IEEE*, and Aviral Shrivastava , *Senior Member, IEEE*

Abstract—Deploying diverse machine learning workloads across hardware targets increasingly relies on multi-level compiler stacks built on infrastructures such as MLIR. Production compilers apply fixed, expert-designed optimization pipelines. Hierarchical progressive lowering makes pass ordering consequential in these stacks. Reordering passes can alter dispatch partitioning and backend routing, which changes the downstream tuning space itself. Substantial workload-dependent performance therefore remains untapped.

We first show empirically that performance varies substantially across compilation pipelines, and that no single fixed pipeline generalizes across models and tensor shapes. We then investigate *why* progressive lowering amplifies phase-ordering effects relative to single-level compilers. Using structural and backend-oriented diagnostics, including dispatch topology, code-generation strategy mix, and instruction proxy metrics, we identify *tuning-space regime shifts*: discrete changes in downstream compilation structure induced by upstream pass reordering.

These findings inform the design of *BO3* (Beyond O3), a legality-aware three-stage reinforcement learning framework for automated pass ordering in IREE. *BO3* combines backend-configuration screening with policy learning over preprocessing-stage pass sequences. It uses graph-based program observations and dynamic action masking to respect structural legality constraints.

Evaluated on 9 workload instances from 7 model families on one pinned x86 CPU core, *BO3* discovers workload-specific pass sequences that outperform the default IREE *-O3* pipeline, achieving a geometric-mean speedup of $5.66\times$. On the ablated workload, graph-structured observations improve solution quality and dynamic legality masking improves rollout robustness.

Index Terms—Compiler pass ordering, MLIR, IREE, machine-learning compilers, reinforcement learning, graph neural networks, autotuning, progressive lowering.

I. INTRODUCTION

MLIR has enabled a broad ecosystem of downstream compiler toolchains spanning machine learning and hardware domains (e.g., IREE, Torch-MLIR, CIRCT, and DSP-MLIR) [1]–[4]. In MLIR-based *machine-learning* compilers, including IREE, compilation relies heavily on *progressive lowering*, where high-level tensor semantics are gradually transformed across multiple abstraction stages to match backend-specific code-generation paths.

This work was supported in part by the National Science Foundation under Grant ACED 2436016 and in part by the Semiconductor Research Corporation (SRC) under Project 3154.

A. Khedkar and A. Shrivastava are with the School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ 85281 USA (e-mail: atharva.khedkar@asu.edu; aviral.shrivastava@asu.edu). M. Kuo was with Arizona State University, Tempe, AZ, USA, when this work was performed and is now with Samsung Semiconductor, San Jose, CA, USA (e-mail: megankuo@asu.edu).

The design choice of a fixed, expert-designed optimization pipeline (e.g., O2, O3) becomes fragile in MLIR-based machine learning compilers. In progressive lowering, each stage exposes a lower-level structure and discards some higher-level information. These choices are not neutral: once information is lowered away, later stages must optimize over the structure that remains. A fixed pipeline, therefore, acts as a fixed information-retention policy across workloads whose profitable lowering trajectories may differ. In other words, early transformations dictate what later stages can fuse, partition, materialize, and route. Pass ordering therefore reshapes the downstream optimization problem itself.

This setting also exposes the limitations of existing approaches. Traditional autotuners are effective when the downstream program structure is largely fixed, and tuning focuses on backend schedules or parameters [5]–[8]. In contrast, upstream pass reordering in IREE can alter the structure presented to later stages. Likewise, heuristics based on immediate IR-local signals are limited because the true performance effect of an early transformation may emerge only after later lowering, through changes in dispatch formation, layout constraints, and backend routing [9], [10].

Our empirical study in Section III shows that these effects are substantial. Across 16 workloads, the best pipeline found through extensive search can outperform the default *-O3* pipeline by a geometric mean of $5\times$. This figure measures the headroom exposed by a 48-hour per-model search, and it is separate from the $5.66\times$ that *BO3* attains under a bounded budget on the 9-instance evaluation suite of Section VII. The two numbers cover different workload sets and different search budgets. Different models, and even different tensor shapes within the same model family, require different pipelines.

To understand *why* pass ordering has such a pronounced impact, we conduct a detailed investigation (Section V). We identify *tuning-space regime shifts*: upstream pass ordering changes that induce discrete, qualitative changes in downstream compilation state. These shifts alter the optimization problem presented to later stages, explaining why local proxy metrics can mislead and why fixed pipelines transfer poorly.

These observations motivate an automation strategy that reasons over long-horizon downstream effects, respects evolving pass legality, and captures non-local program structure. We therefore propose *BO3*, a legality-aware three-stage reinforcement learning framework for pass ordering in IREE. *BO3* combines backend-configuration screening with reinforcement learning over preprocessing stage pass sequences, using graph-based compiler-state observations and dynamic action masking to avoid invalid trajectories.

We evaluate $\mathcal{BO3}$ on 9 workload instances drawn from 7 model families and show consistent improvements over the default $-O3$ pipeline.

All empirical claims in this paper are scoped to a single-core x86 CPU target (AMD Ryzen Threadripper PRO 7955WX, single physically pinned core via `taskset`).

Contributions. This paper makes two main contributions:

- We investigate why pass ordering has a substantial impact on the final performance in multi-level compilers (e.g., IREE) as compared to single-level compilation flow (e.g., LLVM-style pipelines). We identify *tuning-space regime shifts* as an important source of phase-ordering sensitivity in MLIR/IREE progressive lowering.
- We present $\mathcal{BO3}$ (Beyond $O3$): a legality-aware three-stage RL framework that combines backend-configuration screening with graph-based pass search under dynamic legality masking and terminal rewards to discover good workload-specific compilation paths.

We show that on the evaluated CPU suite, $\mathcal{BO3}$ consistently outperforms the default IREE $-O3$ pipeline across 9 workload instances from 7 model families, achieving a $5.66\times$ geometric-mean speedup using only bounded post-training rollouts from the learned policy.

II. BACKGROUND

A. IREE Compilation Stages

IREE is a retargetable MLIR-based compiler that lowers machine-learning workloads to multiple hardware backends, including CPUs and GPUs, through a progressive-lowering compilation flow [11]. IREE applies transformations across several compilation stages, each of which exposes different structural information to later stages. The pipeline consists of three primary stages:

a) Preprocessing stage: This stage operates on high-level whole-program IR and lowers to the dispatch-creation stage. Transformations include layout conversions (e.g., $NCHW^1 \rightarrow NHWC$), padding, attention-folding, and operator canonicalization. This is the stage targeted by our RL agent because these transformations occur before partitioning; they can reshape the program structure observed by later stages.

b) Dispatch-creation stage: IREE partitions the program into *dispatches*. A dispatch is a self-contained executable region that serves as the basic unit for subsequent backend code generation [12]. The number of dispatches, their internal composition, and the dataflow relationships, determine the structure that backend code generation must optimize.

c) Backend code generation: Each dispatch is independently lowered to target-specific code. IREE selects a lowering pipeline for each dispatch based on its structure and a target-specific code-generation configuration we call *knobs* (e.g., tiling, unrolling, vectorization flags).

Because preprocessing precedes dispatch creation, decisions made at this early stage change *what* dispatches are formed and *how* they are lowered.

¹N: batch, C: channels, H: height, W: width. The leftmost symbol is the outermost dimension; the rightmost is the innermost (contiguous) dimension in memory.

B. Named vs. Generic Operation Forms

MLIR organizes transformations around *dialects*, where each dialect defines operations at a particular abstraction level [13]. Compilation proceeds by progressive lowering: high-level operations are gradually rewritten into lower-level dialects, such as `linalg` $\rightarrow$ `vector` $\rightarrow$ `llvm`.

Representation choices also arise *within* a dialect. In `linalg`, the same computation can be expressed as a named op (e.g., `linalg.conv_2d_nhwc_hwcf`) or as a semantically equivalent `linalg.generic` that makes indexing maps, iterator types, and the scalar body explicit in the IR. The pass `linalg-generalize-named-ops` performs this within-dialect form conversion: The two forms encode the same computation, but they expose different information to later compiler stages. The named form carries an explicit operation identity. The generic form makes the iteration structure and indexing behavior explicit. The conversion is asymmetric: a named op can always be generalized, but recovering the named form later requires pattern matching and may fail after other rewrites.

We use this pass in Section V for isolating stage-dependent effects. Applied early, it can expose structure that later rewrites can use, but it removes named-op identity earlier in the pipeline. Applied late, it preserves named-op identity longer, but removes that information closer to downstream routing.

C. Backend Lowering Strategies: Double-Tiling vs. MMT4D

IREE’s CPU backend selects among several dispatch-lowering pipelines. The two that dominate in our analysis are summarized below.

a) Double-tiling (general-purpose path): Double-tiling is a general-purpose path that operates on the original tensor layout. It applies multi-level loop tiling across cache, vector, and reduction granularities. Because layout is not globally transformed in advance, the generated kernel may still require register-level data movement (e.g., shuffles, broadcasts) to satisfy SIMD layout constraints.

b) MMT4D-tiling (specialized path): MMT4D is a specialized path that re-encodes operands into a tiled 4D layout *before* computation. Concretely, a 2D matrix `tensor<MxKxf32>` is reshaped and transposed into `tensor<M1xK1xM0xK0xf32>`, where the outer dimensions ($M1$, $K1$) index tiles, and the inner dimensions ($M0$, $K0$) index elements within a tile [14]. The tiled layout makes tile-local data contiguous in memory and better aligned with SIMD execution. MMT4D can improve kernel efficiency, but it introduces `pack` (2D to 4D) and `unpack` (4D to 2D) operations that materialize the tiled layout. Performance depends on whether these operations fuse with surrounding producers and consumers; unfused `pack/unpack` materialization can add substantial overhead.² When the compiler commits to this tiled-layout path, it may also introduce encoding dispatches that materialize these layout transformations.

²An internal IREE study on MobileBERT-fp32 reported that unfused `pack` and `unpack` operations alone contributed roughly $\sim 33\%$ of end-to-end data-tiling latency, and a `generic + pack` fusion case ran $14.4\times$ slower than the unfused baseline [15].

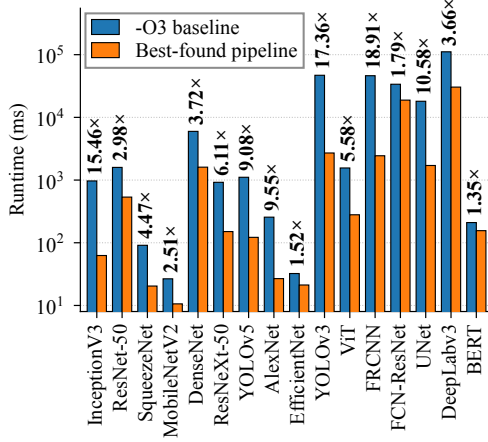


Fig. 1. Performance headroom from workload-specific pass ordering over the default IREE $-O3$ pipeline. Each bar pair shows the $-O3$ baseline runtime (blue) and the best-found pipeline runtime (orange) for a given model; annotations report the speedup ratio. The gap demonstrates that fixed pipelines leave substantial workload-dependent performance untapped.

III. MOTIVATION: FIXED PIPELINES FAIL TO GENERALIZE

Figure 1 shows that across 16 workloads, the best-found pipeline can achieve a geometric-mean speedup of $5\times$ over the default $-O3$. These best-found pipelines were obtained by enumerating approximately 150 available passes across IREE’s compilation stages and searching pass combinations per model over 48 hours. This search establishes available headroom under an unbounded budget, and it is distinct from the bounded-budget result reported for $BO3$ in Section VII. This section uses targeted subsets of these results to demonstrate: (1) different models need different pipelines, and (2) different tensor shapes within the same model family also need different pipelines.

A. Different Models Need Different Pipelines

Figure 2 shows that pass pipelines are not uniformly good or bad across models. Instead, the ranking of candidate pipelines changes substantially across workloads. Each column corresponds to a model, each row to a candidate pass pipeline, and each cell reports speedup over the default $-O3$ baseline. The blue outline marks the best-performing pipeline for each model.

A key observation is that *the identity of the best pipeline changes across models*. For example, $I + \text{peel} + \text{tile}$ is the best pipeline for `frncnn` at $19.91\times$, yet it drops to near-baseline on `resnet50` ($0.98\times$) and fails on `deeplab_513x513`. Conversely, $T \text{ conv} + \text{mask} + \text{tile}$ is best for `deeplab_513x513` at $3.89\times$, but falls to sixth place on `yolo_v5` at only $1.23\times$. Thus, the same passes can be optimal, or even infeasible, depending on the workload.

This behavior goes beyond simple magnitude variation: some pipelines are *structurally infeasible* on certain workloads (marked FAIL in Figure 2), indicating that pass ordering interacts with workload structure beyond simple parameter sensitivity.

$2.84\times$ (r1)	$1.41\times$ (r5)	$1.28\times$ (r5)	$2.27\times$ (r6)	$3.68\times$ (r2)	$0.99\times$ (r6)	T conv+pad+tile
$1.95\times$ (r5)	$2.33\times$ (r1)	$8.72\times$ (r2)	$17.21\times$ (r3)	$0.99\times$ (r4)	$1.04\times$ (r3)	I+tile
$2.15\times$ (r3)	$1.43\times$ (r4)	$9.08\times$ (r1)	$9.44\times$ (r4)	$2.65\times$ (r3)	$1.00\times$ (r4)	CC+I+mask+tile
$0.98\times$ (r6)	$1.54\times$ (r3)	$7.86\times$ (r3)	$19.91\times$ (r1)	FAIL	$1.12\times$ (r2)	I+peel+tile
$2.65\times$ (r2)	$1.38\times$ (r6)	$1.23\times$ (r6)	$2.94\times$ (r5)	$3.89\times$ (r1)	$1.00\times$ (r5)	T conv+mask+tile
$2.14\times$ (r4)	$2.23\times$ (r2)	$7.75\times$ (r4)	$19.20\times$ (r2)	FAIL	$1.36\times$ (r1)	gen+tile+unroll
resnet50	mobilenet v2	yolo v5	frncnn	deeplab 513x513	bert base 128	Model

Fig. 2. Fixed pipelines fail across models. The best-performing pipeline (blue outline) changes across models, and some sequences are invalid on specific workloads (marked FAIL). Shorthand pass bundles: gen=generalize, CC = convert-conv-to-channels-last, T = transpose-convolution, and I = convert-conv2d-to-imag2col.

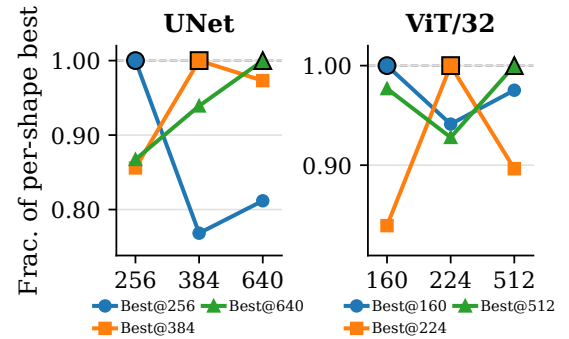


Fig. 3. Fixed pipelines fail across shapes within a fixed model family. Each line corresponds to the best pipeline found for one shape, evaluated on all shapes in that family. Solid markers denote the home shape, where the pipeline is best-found by construction ($1.0\times$). The drop away from the home shape shows that different tensor sizes can induce different profitable pass orderings.

B. Different Tensor Shapes Need Different Pipelines

The previous subsection shows that pass ordering is model-dependent. We now show that this fragility persists even *within the same architecture* when only tensor shape changes. Figure 3 evaluates cross-shape portability by taking the best-found pipeline for each shape and running it on the other shapes of the same model family. Performance is normalized by the best speedup for the *target* shape, so the self-optimum is always $1.0\times$.

Two patterns are immediately visible. First, the best pipeline for one shape is often *not* the best for other shapes. Second, the loss from transferring a pipeline across shapes is non-trivial. For UNet, the pipeline optimized for input size 256×256 loses roughly 23% of the available speedup when evaluated on 384×384 ($0.768\times$ of that shape’s best). It still underperforms on 640×640 ($0.812\times$). For ViT, the best pipeline at 224×224 transfers poorly to 160×160 ($0.839\times$) and also degrades on

512×512 ($0.896\times$). These degradations are substantial even within a fixed model family, ruling out the simpler strategy of tuning one representative shape and reusing that pipeline across other shapes. *Pass ordering in IREE is therefore shape-sensitive as well as architecture-sensitive.*

These results establish that pass ordering in IREE is sensitive to both model architecture and tensor shape.

IV. RELATED WORK

The phase-ordering sensitivity demonstrated in Section III raises a question: can existing automation approaches close the gap? We survey prior work along two axes: (i) the *decision scope* at which the system operates: graph- and operator-level search, loop-nest transformation, or phase ordering and pass selection; and (ii) the *methodology*, primarily distinguishing *search-based autotuners* that perform per-workload search and return workload-specific optimization artifacts (schedules, graph rewrites, or pass sequences) from *learned policies* that train a single model and deploy zero-shot across programs. The subsections below are organized by decision scope; within each, we identify which operating model the prior systems assume. This methodological split motivates *BO3*'s per-workload protocol in Section VII-D. To the best of our knowledge, no prior system targets per-workload search over preprocessing-stage pass orders in IREE before dispatch formation; *BO3* occupies this point in the design space.

A. Graph- and Operator-Level Search

A large body of systems automates performance tuning at the graph or operator level by searching over backend schedules, parameter spaces, or graph substitutions. TASO [16] optimizes DNN computation graphs by applying equivalence-preserving substitutions, which are formally verified against operator specifications and then searched via cost-based backtracking; its primary variable is the graph structure itself. TVM [6] performs operator-level schedule search within human-defined templates, guided by a learned cost model. Ansor [7] removes this dependence on manual templates by automatically generating search spaces through hierarchical program sampling and using a task-level scheduler to prioritize performance-critical operators. MetaSchedule [17] builds tensor-program search spaces from modular schedule rules, exposing stochastic choices such as tiling, fusion, and mapping decisions to users. Halide auto-scheduling [8], [18] addresses the same problem for image-processing pipelines through compute/store-location, fusion, and vectorization decisions expressed in the Halide schedule language. OpenTuner [5], framed as “an extensible framework for program autotuning”, is generic over a user-supplied parameter space.

These per-workload autotuners search a schedule, graph rewrite, or parameter setting for one operator, graph, or workload-target instance, and return that artifact. *BO3* shares this paradigm and applies it to a different optimization variable. TVM, Ansor, and MetaSchedule search kernel schedules inside compute regions that dispatch formation has already fixed. *BO3* searches the preprocessing-stage pass order that determines how those regions are formed in the first place.

The two are therefore complementary. A workload can be tuned by *BO3* at the preprocessing stage and by a schedule-level autotuner afterwards. A head-to-head comparison would not isolate either contribution, so we evaluate *BO3* against baselines in its own action space: legality-constrained random rollout and beam search (Section VII).

B. Loop-Nest Transformation with Learned Search

A second class applies learned search to loop-nest transformation within a single kernel or compute region. PolyGym [19] formulates legal polyhedral loop transformations as a shape-agnostic Markov decision process for loop scheduling, using polyhedral dependence analysis for legality; the paper provides the RL environment with a simple traversal heuristic rather than a deployed learned agent. Pearl [20] integrates a deep RL agent into the Tiramisu polyhedral compiler to optimize general, tensor-manipulating loop nests. Trained on randomly generated programs, it utilizes polyhedral dependence analysis to ensure legality while achieving strong generalization to unseen workloads. MLIR-RL [21] introduces an RL environment for the MLIR Linalg dialect, with a hierarchical multi-action space over tile, interchange, parallelize, vectorize, and unroll, and action masking for divisibility constraints.

These systems differ in operating model. Pearl trains on random programs and targets generalization to unseen loop nests; PolyGym and MLIR-RL provide RL environments evaluated per task. But all three work at the same scope: loop-nest transformation inside an already-formed compute region. Such a learner could run downstream of *BO3*'s preprocessing-stage choices.

C. Phase Ordering and Pass Selection

At the LLVM level, AutoPhase [22] applies RL to phase ordering for HLS, using a 56-dimensional engineered feature vector. Coreset-NVP [23] compresses LLVM's pass space into a fixed coreset of 50 sequences via submodular maximization and learns a GNN-based ranker over ProGraML representations to select candidates for unseen programs under a 45-pass budget, targeting code-size reduction over $-Oz$. CompilerDream [24] uses model-based RL with a compiler world model to learn cross-program LLVM optimization strategies. Beyond LLVM, XLA-RL [25] learns an RL policy over target-independent HLO passes under operation-count proxy rewards, evaluated on GPT-2, BERT, and ResNet graphs benchmark.

MLGO [26] and NeuroVectorizer [27] replace selected in-pass heuristics with trained policies: inlining and register allocation in the case of MLGO; vectorization factor and interleaving in NeuroVectorizer, rather than reordering passes themselves. CompilerGym [28] provides infrastructure, a unified RL environment for LLVM/GCC compiler optimizations.

AutoPhase tunes each benchmark individually (28% over $-O3$), but when trained on random programs and deployed on held-out ones, the gain drops to near 3% on held-out CH-Stone benchmarks. Coreset-NVP, CompilerDream, and XLA-RL commit to the zero-shot transfer setting. This generalization is aided by relatively stable LLVM action spaces, where invalid passes silently become no-ops, allowing a single

fixed coreset to apply across programs. Conversely, MLIR’s progressive lowering structurally rewrites the IR, forcing the legal action set to be dynamically recomputed at every step.

V. WHY PROGRESSIVE LOWERING AMPLIFIES PHASE-ORDERING SENSITIVITY

Because progressive lowering incrementally commits away higher-level structure, early transformations affect which downstream opportunities remain available. Pass ordering therefore acts on optimization outcomes and on the downstream tuning space itself. We refer to these discrete downstream changes as *tuning-space regime shifts*.

Definition 1 (Tuning-space regime shift). An upstream pass-ordering change induces a *tuning-space regime shift* when it produces a discrete, qualitative change in the downstream compilation state that alters the optimization problem presented to later stages. We identify three subtypes, ordered from coarsest to finest granularity:

- 1) **Routing-level:** The set of backend lowering strategies applied to dispatches changes entirely, e.g., dispatches shift from MMT4D to double-tiling.
- 2) **Partition-level:** The compiler redraws dispatch boundaries, altering how much overhead is trapped inside each kernel, while the global routing distribution remains constant.
- 3) **Composition-level:** The internal decomposition or fusion of individual operations diverges, e.g., a softmax is kept monolithic versus being split into independent reduction and elementwise dispatches, while aggregate routing and partitioning appear unchanged.

Composition determines the form of computation in the IR. Partition determines how computations are grouped into dispatches. Routing determines which backend strategy is selected for each dispatch.

This distinction matters because it separates two qualitatively different explanations for pipeline sensitivity. Under a fixed-space view, pass reordering changes code quality only within an otherwise unchanged downstream optimization problem; in that case, differences should be explainable by continuous instruction-level proxies such as compute density, memory pressure, or permutation overhead. Under our regime-shift view, pass reordering changes the downstream problem itself, so even with the same backend knob setting, later stages may receive different dispatch structures, different routing opportunities, or different operator decompositions.

Sections III-A and III-B show that different workloads and shapes demand different pipelines, but they do not explain *what changes structurally* when a pipeline is reordered. Table I addresses this question directly.

To isolate stage-placement effects, we insert `linalg-generalize-named-ops` as a probe pass (Section II-B) at two stages, preprocessing and dispatch creation, while holding the knob fixed. Table I reports three diagnostic layers: *Structure* (dispatch count, encoding dispatches, materialization), *Routing* (backend strategy mix), and *Instruction Profile* (per-dispatch overhead and compute density), with latency as the final outcome. Routing and

partition-level shifts are often visible directly in the table; composition-level shifts require direct IR inspection of specific dispatches.

a) *InceptionV3: routing-level regime shift:* InceptionV3 provides the most direct evidence of a routing-level regime shift induced by stage placement. At baseline and preprocessing, the compiler materializes 47–48 pack/unpack ops and routes ~17% of dispatches through MMT4D, with latency at 968–932 ms. When generalization is delayed to the dispatch-creation stage, all encoding-related dispatches are eliminated, reducing total dispatches from 183 to 127 (this matches the number of encoding dispatches: 56 present at baseline.) Pack/unpack materialization halves, and MMT4D routing disappears entirely.

The instruction-level consequences follow directly. With all dispatches forced into the double-tiling path, per-dispatch permutation operations rise 19× and predicate operations rise 12×: the double-tiling path must perform on-the-fly data rearrangement for dispatches that pre-packed MMT4D layouts would have otherwise served. FMA% rises sharply from 67.0% to 94.3%. However, the higher arithmetic fraction does not translate into lower latency, because it is accompanied by a much more overhead-heavy dispatch regime. Latency regresses from 968 ms to 992 ms. InceptionV3 thus shows a clear chain: stage placement changes dispatch structure, which collapses routing options and leads to a slower backend regime.

b) *ResNet-50: partition-level regime shift:* ResNet-50 shows that regime shifts are not limited to routing-level changes. Routing-level activity is minimal (4 pack/unpack, MMT4D% nearly unchanged). Yet latency drops by 2.4×.

The structural change is visible in dispatch count (33 to 37) and per-dispatch overhead (e.g., Perm./Disp. drops from 445.3 to 328.6). Early generalization restructures the IR so that dispatch-creation produces dispatches with substantially less rearrangement and predication work, even though they are routed broadly to the same strategies.

Late generalization at dispatch creation eliminates all remaining routing activity (0 pack/unpack, 0 encoding dispatches), raises FMA% to 77.8, but barely changes latency or dispatch count. As with InceptionV3, encoding dispatches that support data-tiling are removed; here, the dominant effect is not routing loss (since the workload rarely uses MMT4D) but the missed opportunity for dispatch restructuring that early placement enables.

c) *ViT-224: composition-level regime shift:* ViT-224 reveals that regime shifts can arise at a finer granularity than aggregate strategy-mix percentages can capture. Both non-baseline placements shift the workload from double-tiling-dominant to MMT4D-dominant, with nearly identical routing percentages and comparable materialization operations (pack/unpack ops). Yet early placement achieves 291 ms compared to late placement, 450 ms, a ~55% latency overhead that routing-level and partition-level metrics in Table I do not explain.

IR inspection indicates that the clearest difference lies in how a critical computation is decomposed. Because ViT-224 contains 12 identical transformer blocks, this per-block structural difference is amplified across the full model.

TABLE I

STAGE PLACEMENT INDUCES DISTINCT BACKEND REGIMES. EARLY PLACEMENT CAN RE-ROUTE DISPATCHES TOWARD A DIFFERENT LOWERING STRATEGY, WHILE LATER PLACEMENT CAN TRIGGER OVERHEAD-DOMINANT REGIMES EVEN WHEN COMPUTE-DENSITY PROXIES IMPROVE.

Model	Stage	Structure				Routing		Instruction Profile			
		Lat. (ms)	#Disp.	#Encode Disp.	#pack/unpack	Double%	MMT4D%	Perm./Disp.	Pred./Disp.	Mem./Disp.	FMA%
InceptionV3	Baseline	968	183	56	47	76.0	17.6	20.6	2.4	15.5	67.0
	Preprocessing.	932	183	56	48	77.1	16.4	21.8	2.4	15.6	89.5
	Dispatch-create	992	127	0	23	93.5	0.0	395.1	29.7	23.3	94.3
ResNet-50	Baseline	1595	33	2	4	98.1	1.0	445.3	32.9	68.3	52.7
	Preprocessing.	662	37	3	7	98.1	1.9	328.6	13.4	92.0	45.6
	Dispatch-create	1575	33	0	0	99.0	0.0	409.4	31.1	55.4	77.8
ViT-224	Baseline	480	211	62	20	71.5	28.5	11.9	2.6	5.5	37.6
	Preprocessing.	291	226	63	28	40.5	59.5	25.6	3.7	11.0	31.0
	Dispatch-create	450	214	63	23	39.0	61.0	14.4	2.9	7.0	38.3

FMA%: fraction of emitted arithmetic instructions that are FMAs (e.g., `llvm.instr.fma`).

Perm./Disp.: data-reordering instructions per dispatch (e.g., `llvm.shufflevector`, `llvm.extractelement`). **Pred./Disp.:** masking and predication instructions per dispatch (e.g., `llvm.select`, `llvm.cond_br`). **Mem./Disp.:** load/store instructions per dispatch.

Softmax decomposition. Early generalization decomposes softmax into a reduction dispatch (2364×197) and an element-wise transpose dispatch ($3 \times 197 \times 768$)³. In contrast, late generalization uses a single monolithic `softmax_12 \times 197 \times 197` dispatch, forcing a single tiling strategy to handle structurally heterogeneous work (reduction vs. data movement).

The decomposition difference shows that IR-level composition can remain performance-critical even when aggregate routing statistics appear similar.

Taken together, these 3 cases (InceptionV3, ResNet-50, and ViT-224) instantiate each subtype of tuning-space regime shift from Definition 1. InceptionV3 exhibits a *routing-level* shift: delayed generalization eliminates MMT4D entirely and forces all dispatches into an overhead-heavy double-tiling regime. ResNet-50 exhibits a *partition-level* shift: routing is barely changed, but early generalization restructures dispatch internals to reduce per-dispatch permutation overhead by 26%. ViT-224 exhibits a *composition-level* shift: aggregate routing statistics remain broadly similar, yet softmax is decomposed differently across placements, producing a $\sim 55\%$ latency overhead that is not visible in aggregate metrics.

Section III-A– III-B demonstrates that no single fixed pipeline generalizes across workloads; we attribute this to the **regime shifts** identified in our analysis. An effective automation strategy must therefore reason over long-horizon cross-stage effects, enforce evolving pass legality, and capture non-local program structure. Section VI presents *BO3* around these requirements.

VI. BEYOND O3—THE *BO3* FRAMEWORK

The observations in Section III–V impose three design requirements for an automated pass-ordering framework:

- 1) **Per-workload policy training.** Cross-model and cross-shape sensitivity imply that no fixed pipeline transfers reliably. *BO3* therefore trains one knob-conditioned policy per workload instance.
- 2) **Sparse terminal reward.** Section V shows that stage-local proxies such as FMA% can be anti-correlated with end-to-end latency under regime shifts. We therefore use the

³For max-subtraction and normalization.

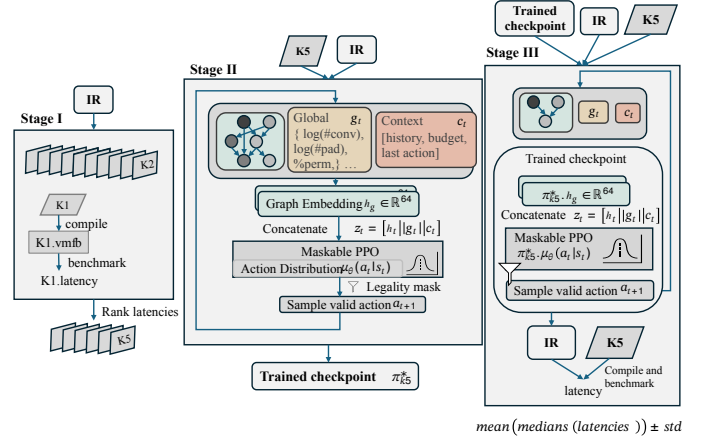


Fig. 4. Overview of *BO3*. **Stage I (Backend Knob Screening):** The top- K backend configurations are benchmarked and retained as anchors. **Stage II (Policy Training):** For each anchor, a Maskable PPO agent learns pass-ordering using GraphSAGE IR embeddings, global and pass-context features, terminating at EVAL action with a log-speedup reward. **Stage III (Candidate Selection):** The overall lowest-latency candidate is selected from 10 rollouts per trained checkpoint.

final measured latency as the primary optimization signal, avoiding dense proxy-based reward shaping.

- 3) **Graph-structured compiler state.** Regime shifts depend on the non-local producer-consumer structure, while pass applicability evolves with the IR. We encode the compiler state as an SSA def-use graph and restrict exploration to currently legal actions.

Figure 4 summarizes the overall workflow. *BO3* operates in three stages: Stage I screens backend configurations, Stage II trains a pass-ordering policy per retained configuration, and Stage III selects final candidates via bounded post-training rollouts.

A. Stage I: Backend Knob Screening

IREE’s backend exposes a discrete set of code-generation parameters (e.g., tiling, unrolling, masking/peeling, vectorization flags), which we collectively refer to as *knobs*. Let $\mathcal{K}_{\text{defs}} = K_1 \times K_2 \times \dots \times K_N$ denote the full knob space, where each K_i is a discrete choice. A specific assignment

$k \in \mathcal{K}_{\text{defs}}$ fixes the backend code-generation configuration for all dispatches in a workload.

Stage I treats knobs as *anchors*. It enumerates all assignments in $\mathcal{K}_{\text{defs}}$, benchmarks them, and retains the top- K configurations ranked by median latency⁴.

Each retained anchor fixes the backend *code-generation configuration*, such as the unrolling flags, masking/peeling behavior, used during backend code generation. The purpose of anchoring knobs is to give the Stage-II policy a stable backend target to prevent code-generation changes from confounding the pass-ordering search.

This decomposition is pragmatic rather than exact: Stage I may prune a knob that would become favorable after pass search reshapes the IR. We evaluate this risk empirically in Section VIII-D2.

Failure modes of staged knob screening: The staged decomposition in *BO3* trades joint optimization over the combined upstream-pass and backend-knob space for computational tractability. It rests on a *stationarity assumption*: the relative ranking of backend knob configurations, screened under the default upstream pipeline in Stage I, remains stable across the mutated IR structures the RL policy generates in Stage II. The regime-shift phenomenon of Definition 1 can violate this assumption in three ways. Stage I may *underrate* a knob whose marginal improvement is negligible while the default pipeline produces small, fragmented dispatches, but which dominates once Stage II fuses those layers into larger dispatches. Stage I may *overrate* a knob that merely compensates for a structural inefficiency in the default pipeline and becomes redundant once Stage II optimizes the upstream IR. Finally, an upstream sequence and a knob may form a tightly coupled, high-performing pair; if neither shows a strong marginal effect in isolation, Stage I prunes the knob and the joint optimum is never reached.

As documented in the phase-ordering literature [29]–[31], early transformations routinely enable or disable the efficacy of later ones. A decoupled knob-then-pass search therefore risks missing the joint optimum, just as decoupled phase ordering is blind to non-monotonic phase interactions. We adopt this decoupled formulation deliberately: it isolates the regime-shift phenomenon under study and enforces a uniform, bounded autotuning budget.

Three structural safeguards reduce these risks. Stage I retains the top- K ($K = 3$) knob anchors rather than a single optimum, and because these anchors differ in tiling, vectorization, and masking/peeling choices, retaining several provides partial coverage against rank inversion and co-design coupling. Stage III samples bounded rollouts across all K knob-conditioned checkpoints and reports the best end-to-end result, so no single anchor determines the outcome. Our cross-anchor stability analysis (Section VIII-D2) shows that on ResNet-50 and ViT-224 the performance gaps between backend knobs remain stable under upstream pass reordering, which indicates that the stationarity assumption holds for the workloads we examined. A systematic check across the full suite remains future work.

The reported $5.66\times$ speedup therefore represents the headroom harvestable within a controlled, bounded deployment budget, not a claim of absolute joint optimality. Quantifying the precise co-design optimization gap over an unbounded search space is left to future work.

B. Stage II: Pass Ordering via Reinforcement Learning

For each retained knob anchor k , Stage II trains a reinforcement learning agent to search over preprocessing-stage pass sequences. We formulate this as an episodic, finite-horizon Markov Decision Process (MDP): $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{M}, \gamma)$, where $\mathcal{S}$ is the state space (compiler IR states), $\mathcal{A}$ is the action space (preprocessing passes), $\mathcal{P}$ is the transition function (applying a pass to the IR), $\mathcal{R}$ is the reward function (based on end-to-end latency), $\mathcal{M}$ is the legality-mask function (restricting actions to currently valid passes), and γ is the discount factor. The environment is instantiated under a fixed backend configuration k from Stage I. We now describe each component of the MDP.

1) *State and Observation:* The environment state s_t is the full compiler IR module after t pass applications. The agent does not observe s_t directly. Instead, it receives an *observation* o_t constructed by extracting a SSA dataflow graph from the IR and encoding it into a fixed-dimension vector.

Concretely, the observation pipeline has three stages:

- 1) **Graph extraction.** From the current IR we extract a directed graph $(\mathcal{V}_t, \mathcal{E}_t)$, where nodes $\mathcal{V}_t$ are IR operations and edges $\mathcal{E}_t$ are SSA def-use dependencies. Each node v is annotated with a feature vector $\mathbf{x}_v \in \mathbb{R}^{27}$ (Section VI-B1a). We also compute a global feature vector $\mathbf{g}_t \in \mathbb{R}^{12}$ that summarizes module-level statistics, and a pass-context vector $\mathbf{c}_t$ encoding remaining budget and most recent action.
- 2) **Graph encoding.** A two-layer GraphSAGE network (Section VI-B1c) aggregates neighborhood information across SSA edges and produces a fixed-dimensional graph embedding $\mathbf{h}_G \in \mathbb{R}^{64}$ via masked mean pooling.
- 3) **Observation vector.** The final observation is the concatenation $o_t = [\mathbf{h}_G \parallel \mathbf{g}_t \parallel \mathbf{c}_t]$, which is passed to the policy and value heads.

a) *Node features:* Each operation $v \in \mathcal{V}_t$ carries a 27-dimensional feature vector $\mathbf{x}_v$ covering structural role and computational behavior: an op-type bucket (1 dim); shape and dynamicity (5 dim: tensor ranks, dynamic-shape indicators, log-scaled operand and result byte sizes); topology and usage (5 dim: fan-out and consumer-type flags, including whether the operation feeds convolutions or matmuls); computational intensity (4 dim: analytical memory footprint, log-scaled FLOPs, arithmetic-intensity proxy); layout and permutation (10 dim: pad, broadcast, and collapse flags plus an 8-dimensional permutation signature); and a generic-op semantic category (2 dim).

The last group compensates for a specific hazard. As Section II-B describes, `linalg-generalize-named-ops` rewrites named operations into `linalg.generic` form and erases the operation identity. Without correction the encoder would see every generalized operation as one op type, losing distinctions such as convolution versus transpose. We recover

⁴We chose $K = 3$ in our evaluations

the semantic identity by inspecting affine indexing maps and iterator types and mapping it back to the corresponding named-op category. The first dimension encodes that recovered category; the second flags whether the op contains a reduction iterator. Both are zero for non-generic ops.

b) *Global features and pass context*: To complement the graph embedding with workload-level and trajectory-level signals, we compute a 12-dim global vector (module-level op-class log-counts, permutation/contraction fractions) and an 11-dim pass-context vector (recent-action one-hot, normalized budget, action history).

c) *Graph encoding with GraphSAGE*: We need a graph encoder that can handle graphs of varying size and topology, since the IR graph changes after every pass, and produces a fixed-dimensional summary suitable for a policy network. We use GraphSAGE [32], an inductive graph neural network that satisfies both requirements.

GraphSAGE learns node representations by iteratively aggregating features from each node's neighborhood. At each layer ℓ , every node v updates its representation by:

$$\mathbf{h}_v^{(\ell)} = \sigma(\mathbf{W}^{(\ell)} \cdot \text{CONCAT}(\mathbf{h}_v^{(\ell-1)}, \text{MEAN}_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell-1)})),$$

where $\mathcal{N}(v)$ is the set of SSA-edge neighbors of v , $\mathbf{W}^{(\ell)}$ is a learnable weight matrix, and σ is the ReLU nonlinearity. The initial representation $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the 27-dimensional node feature vector.

We stack two such layers with hidden dimension 64. This allows each node embedding to incorporate information from its 2-hop SSA neighborhood, capturing local producer-consumer structure such as a `linalg.matmul` consuming the output of a `tensor.pad` derived from a `linalg.conv`.

After message passing, we apply masked mean pooling over all valid nodes to obtain a graph-level embedding $\mathbf{h}_g \in \mathbb{R}^{64}$.

d) *Budgeted graph construction*: To keep inference tractable, we cap observations at 5000 nodes and 20,000 edges. When the IR exceeds these limits, nodes are retained in priority order: compute-heavy Tier-0 operators (e.g., convs, matmuls) first, Tier-1 elementwise and layout operators second, and remaining operations last.

2) *Actions and Legality Masking*: The action space is a discrete vocabulary of $|\mathcal{A}| = 9$ preprocessing-stage passes plus a terminal EVAL action that triggers compilation and benchmarking.

Legality is not static: the set of applicable passes depends on the current IR and the history of previously applied transformations. Without masking, the agent would waste exploration budget on invalid or unproductive actions, substantially slowing training convergence.

We enforce legality through a deterministic mask $\mathbf{m}_t = \mathcal{M}(s_t) \in \{0, 1\}^{|\mathcal{A}|}$ computed from the current IR state. The policy samples only from $\mathcal{A}_{\text{valid}}(s_t) = \{a \mid \mathbf{m}_t[a] = 1\}$. Two representative masking rules: (1) **per-pass count limits** (e.g., masking a pass after two applications per trajectory) and (2) **structural preconditions** (e.g., masking `convert-conv2d-to-img2col` if the IR has no convolution operations). Masking ensures that most explored trajectories are compilable and that the policy's exploration is concentrated on structurally meaningful action sequences.

3) *Reward*: Intermediate benchmarking after every pass is too expensive, and Section V shows that local IR changes are poor surrogates for final latency. We therefore use *sparse terminal reward* with lightweight shaping:

$$r_t = \begin{cases} -\lambda_{\text{err}}, & \text{if compile/runtime error} \\ -\alpha t, & \text{if valid intermediate step } (t < T) \\ \log(L_0/\text{lat}(s_T)), & \text{if successful evaluation } (t = T) \end{cases}$$

L_0 is the baseline ~ 0.3 latency, and $\text{lat}(s_T)$ is the measured latency of the compiled artifact at the terminal step. We set $\lambda_{\text{err}} = 2$ and $\alpha = 0.1$.

The **error penalty** discourages trajectories ending in compile or runtime failure. The **progressive step cost** $-\alpha \cdot t$ imposes a per-step penalty, biasing the policy toward short sequences: the fifth pass costs $5 \times$ as much as the first (-0.5 vs. -0.1), discouraging redundant trailing actions. The **terminal log-speedup** is the primary signal; the logarithm reduces scale asymmetry between large slowdowns and speedups, stabilizing PPO updates.

C. Policy Optimization with Maskable PPO

We need a policy-gradient method that is (i) stable under the sparse, high-variance reward signal from end-to-end latency measurement, and (ii) compatible with dynamic action masking. Proximal Policy Optimization (PPO) [33] satisfies both: it constrains each parameter update to a trust region around the current policy, preventing excessively large updates caused by noisy reward signals.

a) *Network architecture*: The policy and value networks share a common feature extractor: the GraphSAGE encoder described above produces the observation vector $\mathbf{o}_t = [\mathbf{h}_g \parallel \mathbf{g}_t \parallel \mathbf{c}_t]$. This vector is then passed through two separate linear heads: **Policy head**: $\mathbf{o}_t \rightarrow \mathbb{R}^{|\mathcal{A}|}$ (raw logits over the action vocabulary). Logits of masked actions are set to $-\infty$ before softmax, producing a valid probability distribution $\mu_\theta(a_t \mid s_t, \mathbf{m}_t)$. **Value head**: $\mathbf{o}_t \rightarrow \mathbb{R}^1$ (scalar state-value estimate $\hat{V}(s_t)$, used for advantage computation).

Sharing the encoder between policy and value heads reduces parameter count and ensures that both heads benefit from the same learned graph representation.

b) *Training objective*: PPO maximizes the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(w_t(\theta) \hat{A}_t, \text{clip}(w_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right],$$

where $w_t(\theta) = \mu_\theta(a_t \mid s_t) / \mu_{\theta_{\text{old}}}(a_t \mid s_t)$ is the probability ratio between the current and previous policy and $\hat{A}_t$ is the generalized advantage estimate. Our rewards come from end-to-end latency measurements, which are sparse and noisy. The clipped ratio bounds how far one update can move the policy, so a few high-variance rollouts cannot drive an overly aggressive update.

c) *Masking integration*: Because masked logits are set to $-\infty$ before softmax, invalid actions receive zero probability mass. Gradients are computed only over valid actions, so the policy never receives signal to explore illegal trajectories. This is the Maskable PPO formulation [34], which integrates action masking directly into the PPO update without requiring separate penalty terms.

d) *Training configuration*: Training hyperparameters are specified in Section VII-D. Under the one-knob-one-policy paradigm, each policy specializes to a fixed backend configuration k , focusing the learner on pass sequences compatible with that configuration.

D. Stage III: Post-Training Candidate Selection

Stage II outputs a trained policy checkpoint $\mu_{\theta,k}^*$ for each retained knob anchor. Final candidate selection is decoupled from training to avoid conflating policy quality with best-case training-time luck.

The procedure is as follows: **1) Rollout sampling**. For each trained checkpoint, we sample $N_{\text{ep}} = 10$ post-training rollouts by stochastically drawing actions from the masked policy distribution $\mu_{\theta,k}^*(a | s, \mathbf{m})$ until the agent emits EVAL or the trajectory terminates. Each rollout produces a candidate pass sequence σ_i . **2) Compilation and validation**. Each σ_i is compiled end-to-end under the corresponding knob anchor k . Sequences that produce compile errors or runtime failures are discarded. **3) High-fidelity re-evaluation**. All valid σ_i are benchmarked using the same latency protocol described in Section VII-D. The candidate with the lowest mean latency across all knob anchors is reported as the final result.

This design separates *what the policy learned* (Stage II) from *how candidates are selected* (Stage III).

VII. EVALUATION

A. BO3's Operating Model: A Per-Workload Offline Autotuner

We position BO3 within the established autotuning literature to make its intended use explicit and to motivate the train/evaluation protocol that follows.

BO3 is a per-workload offline autotuner. Given a target workload (model + input shape + hardware target), BO3 produces a compilation path for that workload within a compute budget. BO3 outputs a trained policy together with the resulting pipeline, not a model expected to generalize zero-shot across workloads.

This aligns with established per-workload autotuners, including TVM/AutoTVM [6], [35], Ansor [7], OpenTuner [5], and the Halide auto-scheduler [8], [18]. For instance, Ansor explicitly adopts a per-workload formulation, allocating up to $1000 \times n$ measurement trials per DNN (where n is the number of constituent subgraphs) [7], while AutoTVM optimizes at the granular level of (operator, shape, target) tuples. Coreset-NVP [23] further formalizes this distinction by contrasting its zero-shot generalization against "adaptive search algorithms" that tune programs over multi-hour horizons, thereby establishing them as two fundamentally distinct design points.

The regime shift documented in Section III motivates a per-workload optimization protocol. Because upstream pass reordering induces discrete changes in downstream dispatch partitioning and backend routing, the optimization landscape is workload-conditional: a policy trained on instance A and deployed zero-shot on instance B would face a different observation distribution, and under dynamic legality masking, a different action space, changing both the decisions available

to the policy and the rewards they yield. Whether such zero-shot transfer succeeds is therefore an empirical question rather than a property that can be assumed. BO3 makes no such assumption: it adopts the per-workload protocol.

This per-workload protocol applies uniformly to every method we compare. BO3, random rollout, and beam search all operate on the same (model, shape, target) instance under an identical search budget.

B. Experimental Setup

All experiments run on an AMD Ryzen Threadripper PRO 7955WX (Ubuntu 22.04.5 LTS) using iree-base-compiler/runtime version 3.9-rc (2025-11-25). RL environments use PyTorch 2.9.1, SB3-Contrib 2.7.0, and Gymnasium 1.2.0. Models originate from PyTorch and are exported to ONNX (opset 18). All benchmarking uses the IREE local-sync device, pinned to a single physical core via taskset. Identical benchmarking tools and latency-aggregation procedures are used across all configurations. All experimental claims, including the $5.66\times$ geometric-mean speedup and the regime-shift case studies, refer to this single-core x86 CPU configuration.

C. Benchmarks and Tasks

Our evaluation suite comprises 7 representative deep learning model families chosen for their architectural heterogeneity, including standard convolutional networks (AlexNet, ResNet-50, DenseNet), natural language processing models (BERT-Base), Vision Transformers (ViT-224), object detection models with multi-scale slicing (YOLOv5), and high-resolution encoder-decoder architectures (UNet, evaluated at three spatial resolutions: 256×256 , 384×384 , and 640×640).

For each model, Stage I identifies the top-3 backend knob configurations. Stage II then trains an independent knob-conditioned policy checkpoint for each retained configuration.

Figure 1 (Section III) reports performance headroom over BO3 across 16 workloads to establish the broad impact of pass ordering. We distill this to a 9-instance evaluation suite that spans CNNs, Transformers, one-stage detection, and encoder-decoder dense prediction. The retained instances are AlexNet, ResNet-50, DenseNet, BERT-Base, ViT-224, YOLOv5, and UNet at 256, 384, and 640. InceptionV3, SqueezeNet, MobileNetV2, ResNeXt-50, EfficientNet, and YOLOv3 are structurally covered by these instances. FRCNN, DeepLabV3, and FCN-ResNet are omitted because they exceed the uniform search budget under this protocol. This scoping also enforces *search-budget uniformity*: every instance receives an identical, bounded Stage-II RL rollout and iteration budget, keeping comparisons across models well-controlled. Results are shown in Table II.

D. Evaluation Protocol and Fairness Controls

a) *Training (Stage II)*: Each Stage II policy trains for $T_{\text{steps}} = 3,000$ environment steps with PPO rollout buffer $n_{\text{steps}} = 50$ and entropy coefficient 0.001; all other hyperparameters follow SB3-Contrib Maskable PPO defaults.

b) *Candidate selection (Stage III)*: From each trained checkpoint we sample $N_{\text{ep}} = 10$ post-training rollouts. Each candidate sequence terminates with an EVAL action, is compiled, and pre-benchmarked 10 times during selection. The lowest-latency candidate is selected for high-fidelity evaluation and is reported as **RL Found** in Table II.

c) *Latency measurement*: The selected candidate is re-evaluated using two-level aggregation: 10 independent invocations, each consisting of 5 repetitions whose median is recorded. We report $\text{mean}(\text{medians}) \pm \text{std}(\text{medians})$ across the 10 medians. This mirrors standard practice in production autotuners such as Ansor.

d) *Random-rollout reference*: For each task, the random baseline samples actions uniformly from the same pass vocabulary, legality mask, and per-pass limits as $\mathcal{BO3}$. Under a matched budget of $B_{\text{eval}} = 610$ attempted rollouts (the ~ 600 training rollouts plus 10 evaluation rollouts), the best valid random-rollout latency is reported.

e) *Rollout robustness (win rate)*: The win rate is the fraction of sampled candidates achieving lower latency than the default $-\text{O3}$ pipeline.

VIII. EXPERIMENTAL CLAIMS

A. Claim 1: $\mathcal{BO3}$ Outperforms IREE $-\text{O3}$ with a $5.66\times$ Geometric-Mean Speedup Across All 9 Workload Instances

Across diverse workloads and backend configurations, $\mathcal{BO3}$ discovers workload-specific pass sequences that consistently improve over the default IREE $-\text{O3}$ pipeline.

Experiment. For each of the 27 tasks, Stage II produces a trained policy checkpoint. We sample a bounded number of post-training rollouts and report the best valid candidate under a common high-fidelity latency protocol (Section VII-D). Table II shows that, across the 9 workload instances, $\mathcal{BO3}$ improves over $-\text{O3}$ on every instance. Aggregating each workload by its best RL-found result across the retained knob anchors yields a geometric-mean speedup of $5.66\times$.

The gains are strongly workload-dependent, consistent with the regime-shift phenomenon. Two patterns emerge: **1) Knob-dominant workloads.** On ViT-224, most of the gain comes from Stage I: the best knob anchor alone reaches ~ 320 ms ($4.9\times$ over $-\text{O3}$), and Stage II provides a further $1.11\times$ improvement to 287 ms. **2) Pass-ordering-dominant workloads.** On the remaining models, pass ordering produces the dominant gain. On UNet-256, the best knob anchor reduces latency by only $\sim 0.4\%$, yet the RL-found sequence under the same anchor achieves $9.6\times$ speedup. UNet-384, UNet-640, ResNet-50, and YOLOv5 exhibit the same pattern at varying scale. These gains are associated with structural IR changes that reroute or restructure downstream dispatches.

Conclusion. $\mathcal{BO3}$ consistently improves over IREE $-\text{O3}$. The relative contributions of Stage I and Stage II are workload-dependent, confirming the staged design.

B. Claim 2: The Gains Are Not from Search Budget Alone

$\mathcal{BO3}$'s advantage over the random-rollout baseline holds under a matched rollout budget. On harder tasks, learned policy guidance is necessary to reliably reach high-performing

regimes. On the ablated workload, graph-structured observations improve peak performance and legality masking improves rollout reliability.

Experiment 2a: RL vs. random rollout. For each task, a random baseline samples actions uniformly from the same environment settings.

On shallow tasks (YOLOv5, BERT-Base), both methods find near-identical candidates: the profitable region is easy to hit by chance. On harder tasks, the gap is large: on UNet-256 Top-1 (2336 ms random vs. 263 ms RL), UNet-384 Top-2 (5667 vs. 683 ms), and UNet-640 Top-1 (14819 vs. 1708 ms). Random search misses entering the high-performing regime on these workloads.

Experiment 2b: RL vs. beam search. Beam search expands all legal actions from each beam entry and retains the top- B partial sequences ranked by compiled latency. We use $B=2$ to match $\mathcal{BO3}$'s per-step branching factor; wider beams would require proportionally more compilations, which we leave to future work.

On BERT-Base, UNet-256, and YOLOv5, beam search ties the RL candidate; the best pass sequence is short (1–2 passes), and the first-step-best action leads directly to the dominant regime. On ResNet-50 (660 ms to 558 ms), ViT-224 (333 ms to 287 ms), and UNet-640 (1879 ms to 1708 ms), RL reaches lower-latency regimes.

Experiment 2c: component ablation. A natural question is which components of $\mathcal{BO3}$ account for its advantage over beam search.

We compare component variants on UNet-640 Top-1, with one trained policy per configuration (Table III). Removing legality masking preserves the best-found latency at 1708 ms, but lowers the rollout win rate from 100% to 70%, indicating that legality constraints primarily improve reliability rather than peak performance. Removing the graph encoder is substantially more harmful: the best-found latency degrades to 1876 ms and the win rate drops to 40%, which indicates that graph-structured state matters most on this task. When both components are removed, the policy reaches 1879 ms, matching beam search, and the win rate falls further to 30%. We further isolate the contribution of engineered node and edge features by replacing the full observation with a flat vector [pass history || remaining step budget], removing all graph-structured input while keeping the RL framework intact (last row, Table III). Best-found latency degrades to **1879 ms** and the rollout win rate falls to 10%, on par with beam search. The advantage of full $\mathcal{BO3}$ therefore comes from combining legality-aware action masking with learned graph representations. Sampling pass orders alone reproduces beam-search-level behavior. On this task, the 9.1% latency reduction from beam search to full $\mathcal{BO3}$ (1879 ms $\rightarrow$ 1708 ms, a $1.10\times$ speedup) is primarily attributable to these learned components.

Conclusion. On shallow tasks, simple search strategies are sufficient to reach the profitable regime. On harder tasks, $\mathcal{BO3}$ reaches lower-latency regimes than beam search. On the ablated task, legality masking raises the rollout win rate and the graph encoder accounts for most of the latency gap to beam search. Repeating the ablation across model families and seeds remains future work.

TABLE II
MAIN EVALUATION RESULTS ACROSS 27 KNOB-CONDITIONED OPTIMIZATION TASKS (9 WORKLOAD INSTANCES $\times$ 3 KNOB ANCHORS).

Model	O3 (ms)	Knob Tuple (U, T, P, V)	Knob only	Random (ms)	RL Found (ms)	Random Spd.	RL Spd.	Rollout win rate
AlexNet	256 $\pm$ 1.91	Top-1 (T, T, m, F)	246 $\pm$ 0.51	55.1 $\pm$ 0.49	20.5 $\pm$ 0.05	4.65 $\times$	12.49$\times$	100%
		Top-2 (T, T, m, T)	246 $\pm$ 0.61	234 $\pm$ 0.44	20.5 $\pm$ 0.03	1.09 $\times$	12.49 $\times$	80%
		Top-3 (T, F, m, F)	247 $\pm$ 0.51	207 $\pm$ 0.32	23.7 $\pm$ 0.05	1.23 $\times$	10.80 $\times$	100%
ResNet-50	1592 $\pm$ 4.45	Top-1 (F, F, m, T)	1548 $\pm$ 1.54	659 $\pm$ 3.77	558 $\pm$ 2.17	2.42 $\times$	2.85$\times$	100%
		Top-2 (T, F, m, T)	1549 $\pm$ 3.43	661 $\pm$ 2.79	560 $\pm$ 2.51	2.41 $\times$	2.84 $\times$	90%
		Top-3 (F, F, m, F)	1555 $\pm$ 1.02	660 $\pm$ 2.87	564 $\pm$ 2.90	2.41 $\times$	2.82 $\times$	70%
BERT-Base	210 $\pm$ 0.61	Top-1 (F, T, p, T)	190 $\pm$ 0.50	156 $\pm$ 0.79	156 $\pm$ 0.34	1.35 $\times$	1.35$\times$	100%
		Top-2 (F, T, n, T)	190 $\pm$ 0.65	157 $\pm$ 1.02	157 $\pm$ 0.42	1.33 $\times$	1.33 $\times$	100%
		Top-3 (F, T, n, F)	191 $\pm$ 1.02	158 $\pm$ 0.62	158 $\pm$ 0.59	1.33 $\times$	1.33 $\times$	100%
UNet: 256x256	2524 $\pm$ 6.73	Top-1 (T, T, n, F)	2513 $\pm$ 5.91	2336 $\pm$ 1.80	263 $\pm$ 0.70	1.08 $\times$	9.60$\times$	100%
		Top-2 (T, T, n, T)	2516 $\pm$ 7.11	263 $\pm$ 0.94	264 $\pm$ 0.46	9.60 $\times$	9.56 $\times$	100%
		Top-3 (T, T, m, F)	2515 $\pm$ 3.25	2337 $\pm$ 6.01	264 $\pm$ 0.95	1.08 $\times$	9.56 $\times$	100%
UNet: 384x384	5672 $\pm$ 10.10	Top-1 (T, F, n, F)	5651 $\pm$ 9.75	4543 $\pm$ 1.57	669 $\pm$ 3.37	1.25 $\times$	8.48$\times$	100%
		Top-2 (T, T, n, T)	5688 $\pm$ 16.90	5667 $\pm$ 13.80	683 $\pm$ 3.12	1.00 $\times$	8.30 $\times$	100%
		Top-3 (F, T, n, F)	5683 $\pm$ 12.60	5899 $\pm$ 2.63	683 $\pm$ 2.62	0.96 $\times$	8.30 $\times$	100%
UNet: 640x640	18067 $\pm$ 29.00	Top-1 (T, F, p, T)	16315 $\pm$ 18.20	14819 $\pm$ 4.01	1708 $\pm$ 4.25	1.22 $\times$	10.58$\times$	100%
		Top-2 (F, T, p, T)	16365 $\pm$ 23.00	15893 $\pm$ 8.05	1968 $\pm$ 1.90	1.13 $\times$	9.18 $\times$	90%
		Top-3 (T, T, p, T)	16366 $\pm$ 15.90	14636 $\pm$ 2.57	1971 $\pm$ 2.76	1.23 $\times$	9.17 $\times$	100%
YOLOv5	1108 $\pm$ 3.57	Top-1 (T, F, p, F)	1061 $\pm$ 3.08	993 $\pm$ 3.57	141 $\pm$ 0.50	1.12 $\times$	7.86$\times$	100%
		Top-2 (T, F, p, T)	1061 $\pm$ 3.53	1097 $\pm$ 2.59	141 $\pm$ 0.56	1.01 $\times$	7.86 $\times$	100%
		Top-3 (F, F, p, T)	1056 $\pm$ 4.20	142 $\pm$ 0.80	141 $\pm$ 0.60	7.80 $\times$	7.86 $\times$	100%
ViT-224	1559 $\pm$ 2.60	Top-1 (F, T, p, T)	319 $\pm$ 1.24	321 $\pm$ 0.965	287 $\pm$ 1.93	4.86 $\times$	5.43$\times$	80%
		Top-2 (T, T, p, T)	320 $\pm$ 0.94	320 $\pm$ 0.68	291 $\pm$ 1.24	4.87 $\times$	5.36 $\times$	80%
		Top-3 (F, T, p, F)	320 $\pm$ 0.57	293 $\pm$ 1.23	291 $\pm$ 1.10	5.32 $\times$	5.36 $\times$	100%
DenseNet	5980 $\pm$ 4.14	Top-1 (T, T, p, F)	4881 $\pm$ 4.65	4813 $\pm$ 4.65	1764 $\pm$ 1.79	1.24 $\times$	3.39$\times$	100%
		Top-2 (F, T, p, T)	4877 $\pm$ 4.23	4768 $\pm$ 4.91	1765 $\pm$ 2.14	1.25 $\times$	3.39 $\times$	100%
		Top-3 (T, T, p, T)	4888 $\pm$ 7.17	4881 $\pm$ 2.14	1767 $\pm$ 3.71	1.22 $\times$	3.38 $\times$	100%

O3: default compiler pipeline. **Knob only**: latency under the same knob anchor without additional pass search.

Random / RL Found: best valid candidate from legality-constrained random rollout and from bounded post-training rollout sampling of the trained knob-conditioned policy, under the same evaluation budget.

Random/RL Spd.: speedup over $-O3$. **Rollout win rate**: fraction of post-training policy rollouts that beat $-O3$ under the same knob anchor.

Knob tuples (U, T, P, V) denote Unroll, Tile, Vector PProc (m = mask, p = peel, n = none), and Vectorization.

TABLE III
COMPONENT ABLATION ON UNET-640 TOP-1. REMOVING THE LEARNED COMPONENTS REDUCES $\mathcal{BO}3$ TO BEAM-SEARCH-LEVEL LATENCY. THE FLAT-VECTOR ROW REPLACES THE GRAPH OBSERVATION WITH [PASS HISTORY || REMAINING STEP BUDGET].

Configuration	Best lat. (ms)	Win rate
Full $\mathcal{BO}3$	1708	100%
No legality masking	1708	70%
No graph encoder	1876	40%
No masking + no encoder	1879	30%
Flat vector observation	1879	10%
Beam search	1879	—

C. Claim 3: A Structural Regime Shift Accounts for the UNet-640 Gain

On UNet-640, the improvement $\mathcal{BO}3$ finds is associated with discrete changes in dispatch structure, the regime-shift phenomenon characterized in Section III. We examine this mechanism in detail for one workload instance. We do not claim that the same mechanism accounts for the gain on every instance in Table II.

Experiment. We examine UNet-640 under the Top-1 knob anchor, which does not enable data-tiling-related backend paths (no encoding dispatches or MMT4D routing are present). This makes UNet-640 a useful complement to the analysis

in Section III. Whereas InceptionV3 demonstrated a *routing-level* regime shift through the elimination of MMT4D, UNet-640 shows how routing and composition move together within the remaining two-strategy space, where the choice for each dispatch lies between the double-tiling path and the default tiling path.

We compare three search procedures: random rollout, beam search, and policy-guided rollout, at three diagnostic levels: dispatch-level structure, instruction-level profile, and hardware-counter measurements.

Structural analysis. Under this anchor the backend routes every dispatch to one of two CPU lowering strategies, the specialized double-tiling path or the default tiling path. The three procedures produce qualitatively different dispatch structures, and they also differ in how dispatches divide between these two paths. Double-tiling covers 75.5% of dispatches under beam search and 88.7% under $\mathcal{BO}3$ (Table IV); the remainder use the default tiling strategy in both cases. The layout change therefore moves roughly 13 percentage points of dispatches onto the specialized path. Table IV summarizes the dispatch-type breakdown for the two candidates that apply img2col decomposition (beam and $\mathcal{BO}3$).

Random rollout (T + EVAL) transposes convolutions but barely changes dispatch structure: dispatch count rises, and latency drops modestly from 18067 ms to 14819 ms.

Beam search applies $\mathbb{I}$ (img2col), decomposing monolithic

convolution dispatches into explicit transpose and matmul components. The dispatch count rises to 186 with 47 matmul dispatches and 47 generic data-rearrangement dispatches. Latency drops to 1879 ms, attributed to dispatch restructuring, with the strategy mix close to the baseline (77.4% to 75.5% double-tiling).

BO3 finds `canonicalize+CC+I+EVAL`. The two preceding passes normalize tensor expressions and convert the data layout from NCHW to NHWC before `img2col`. The total dispatch count is comparable, and the two candidates share largely the same semantic work. The difference lies in how the compiler linearizes and tiles the iteration space of each dispatch.

Layout propagation. IR comparison shows that `CC`'s layout conversion propagates through the entire program. Nearly every dispatch shows permuted dimensions: a broadcast dispatch shaped 102400×64 under NCHW layout becomes 64×102400 under *BO3*; a generic re-arrangement dispatch shaped $320 \times 320 \times 7 \times 7 \times 3$ becomes $3 \times 7 \times 7 \times 320 \times 320$; matmul shapes are also reordered. Because dimension order determines which axis is innermost in memory, this reordering changes the contiguous dimension, the innermost loop, the vectorized dimension, and how working sets land on pages and cache lines, for every dispatch in the program.

Decomposition changes. Table IV shows that the layout change also alters how the compiler expresses movement and re-arrangement works. *BO3* produces more explicit transpose dispatches and fewer broadcast dispatches. Under the beam-found sequence, reductions are folded into generic dispatches (e.g., `generic_64x80x80`) whereas *BO3* preserves the named form (e.g., `reduction_128x80x80x64`). These dispatch-type shifts are best understood as *symptoms* of the layout change. They reflect different lowering paths for the same semantic work, and they do not by themselves indicate which variant executes faster. The hardware counters below characterize the execution-level behavior of the two candidates.

Hardware-counter analysis (Table V). The counter data characterizes how the two candidates execute. TLB misses drop $6\times$. The pervasive dimension reordering changes the memory traversal pattern for every dispatch; this pattern is consistent with better translation locality under NHWC traversal on this microarchitecture. Packed 256-bit SIMD retirement rises from 813M (beam) to 22B (*BO3*), a $27\times$ increase. Under NCHW the channel dimension is not innermost, so populating SIMD registers requires gathering across channels. Under NHWC channels are innermost, so contiguous memory spans align with the SIMD lane width. Not-complete events drop from 73B to 57B, and IPC rises from 1.15 to 1.6.

BO3 retires *more* total instructions than beam (156B vs. 124B), yet completes in fewer cycles and lower wall time. The additional instructions are therefore cheaper than the stall overhead they avoid.

Mechanistic summary. The measurements are consistent with the following mechanism: (1) **Layout change:** `canonicalize+CC` convert the data layout from NCHW to NHWC. (2) **Dispatch decomposition:** The layout propagates, reorienting iteration spaces and altering data-movement

TABLE IV
UNET-640 CASE STUDY UNDER THE TOP-1 KNOB ANCHOR. UPPER BLOCK: DISPATCH STRUCTURE AND INSTRUCTION PROFILE. LOWER BLOCK: DISPATCH COMPOSITION BY SEMANTIC TYPE FOR THE TWO `IMG2COL` CANDIDATES.

Method	Lat. (ms)	#Disp.	Double%	Mem./Disp.	Perm./Disp.	FMA%
O3 / baseline	18067	75	77.44	21.47	20.29	85.71
Random rollout	14819	77	81.12	22.38	30.64	78.32
Beam search	1879	186	75.49	27.75	45.79	92.93
<i>BO3</i>	1708	183	88.74	52.62	147.26	96.69

Method	Matmul	Generic	Memcpy	Broadcast	Transpose	Reduction
Beam	47	47	44	41	6	0
<i>BO3</i>	44	44	42	35	14	3

TABLE V
EXECUTION-SIDE SUPPORT FOR THE UNET-640 CASE STUDY. RL FURTHER IMPROVES ON BEAM BY REDUCING TRANSLATION/STALL PRESSURE WHILE GREATLY INCREASING PACKED SIMD ACTIVITY.

Method	CPU cycles	Instructions	IPC	TLB misses	Not-complete	Pack256 Uops
Random rollout	368B	1T	2.72	311M	198B	367M
Beam search	108B	124B	1.15	6B	73B	813M
<i>BO3</i>	96B	156B	1.6	1B	57B	22B

expression. (3) **Memory behavior:** NHWC places channels innermost, aligning contiguous memory spans with SIMD lanes. (4) **Hardware counters:** TLB misses drop $6\times$, packed-SIMD activity rises $27\times$, stall pressure falls, and IPC rises. (5) **Execution efficiency:** *BO3* completes in 11% fewer cycles with a faster runtime, dominating the instruction count. Per-dispatch proxies do not reflect *BO3*'s performance gains: *BO3* actually *increases* per-dispatch permutation and memory counts (Table IV). The speedup instead tracks the layout decision. Decomposing the same computation under a different dimension order changes hardware-level execution in a way that dispatch-level proxies do not expose.

This account has limits that the present experiment does not resolve. The two candidates differ by two passes, `canonicalize` and `CC`, and we run no ablation that separates their individual contributions. The candidates also differ in dispatch composition, with more transpose and reduction dispatches and fewer broadcast dispatches under *BO3* (Table IV), and in the share of dispatches reaching the double-tiling path. Each of these offers a further route to the observed counters. We therefore report the layout change as the mechanism most consistent with the evidence, and we leave a controlled single-pass attribution to future work.

Conclusion. This case study complements the analysis in Section III. InceptionV3 exhibited a routing-level shift; ResNet-50 exhibited a partition-level shift; ViT-224 exhibited a composition-level shift; UNet-640 combines routing- and composition-level effects. With MMT4D unavailable under this anchor and dispatch counts similar (183 vs. 186), a program-wide layout change moves roughly 13 percentage points of dispatches from the default tiling path onto the double-tiling path and produces qualitatively different execution behavior, including $27\times$ higher SIMD utilization and $6\times$ fewer TLB misses. The common pattern is that profitable pass ordering reshapes how downstream stages represent, partition, and lower the program, producing effects that stage-local proxies do not predict.

D. Claim 4: *BO3 Is Robust, Consistent, and Practical*

To establish *BO3* as a viable deployment-time autotuner, we evaluate three operational properties: robustness (reliable success rather than rare lucky rollouts), consistency across the three-stage decomposition, and practical offline search cost.

1) *Rollout Robustness*: The rollout win rate in Table II is high across tasks. The mean win rate over all 27 tasks is 95.93%. On higher-gain workloads, including the UNet family, YOLOv5, and AlexNet, win rates are consistently 80–100%. This indicates that the trained policy reliably produces candidates that outperform *-O3*, rather than succeeding only through occasional outliers.

2) *Cross-Anchor Stability*: We next test the structural consistency of the three-stage decomposition by cross-executing learned sequences under different knob anchors. If the factorization is meaningful, upstream pass reordering should have limited ability to rescue a poor backend knob configuration.

On ViT-224, sequences learned under the Top-1 and Worst-1 both reach 287–291 ms when evaluated under the Top-1 knob, but remain bottlenecked at approximately 1620–1630 ms under the Worst-1 knob. The resulting $5.7\times$ gap is not bridged by pass ordering. On ResNet-50, the policy converges to the same pass sequence under both anchors, yet the Top-1 anchor reaches 558 ms while the Worst-1 anchor reaches 600 ms. Here again, the 7% residual gap is not removed by pass reordering. Together, these results indicate that the remaining gap is a property of the backend configuration rather than the upstream sequence, supporting the proposed three-stage decomposition. This evidence is limited to two complementary workloads, so subtler knob–pass interactions on other workloads remain future work.

3) *Practical Cost*: Stage II requires a geometric-mean wall-clock time of 5.83 hours per task, with per-task times ranging from 2.8 to 12.6 hours. *BO3* is therefore best viewed as an offline deployment-time autotuning method, with cost comparable to typical tensor-program autotuning budgets [6], [7]. Anso, for example, allocates up to $1000 \times n$ measurement trials per network, where n is the number of constituent subgraphs [7].

IX. CONCLUSION

In MLIR-based compilers with progressive lowering, pass ordering can induce discrete downstream regime shifts that alter dispatch partitioning, backend routing, and the effective tuning space presented to later stages. The effect extends past code quality within a fixed compilation regime. We observe these shifts at the routing, partition, and composition levels. We showed empirically that these shifts make pass ordering sensitive to model architecture, tensor shape, and lowering stage, and that standard local proxies such as compute density can be anti-correlated with end-to-end performance under regime transitions.

BO3 is designed around these observations: it uses per-workload policy training instead of fixed pipelines, sparse terminal latency reward instead of dense proxy guidance, and graph-structured SSA observations with dynamic legality masking instead of flat local state. Its three-stage decomposition further aligns decision variables with what is actually

observable at each stage of progressive lowering. Across 9 workload instances from 7 model families, *BO3* achieves a $5.66\times$ geometric-mean speedup over the default *-O3* pipeline.

Our single-core x86 CPU evaluation serves as a controlled first target: on a single pinned core, latency differences reflect compiler-induced changes in dispatch structure and backend routing, with runtime scheduling effects largely removed. Extending the evaluation to multi-core CPU and GPU backends remains future work. Their dispatch-formation, occupancy, and kernel-scheduling regimes differ from the CPU path, so the routing-, partition-, and composition-level regime shifts characterized here would manifest differently. Our comparison against random and beam search further shows that learned policy guidance is most valuable on the workloads with the largest headroom, where short-horizon search settles for a worse regime. Characterizing the intermediate states along those trajectories requires trace-level analysis that we leave to future work.

ACKNOWLEDGMENT

The authors used generative AI tools to edit the language of author-written text and to assist with bug fixing in the experimental code. The design of the compilation framework, the experimental methodology, and the analysis are the authors' own work. The authors reviewed and verified all AI-assisted output and take full responsibility for the content of this article.

REFERENCES

- [1] "Users of mlir," <https://mlir.llvm.org/users/>, accessed 2026-03-05.
- [2] "Torch-mlir project," <https://github.com/llvm/torch-mlir>, accessed 2026-03-05.
- [3] "Circ tutorial / overview," <https://discourse.llvm.org/t/circ-tutorial/80442>, accessed 2026-03-05.
- [4] A. Kumar, A. Khedkar, H. So, M. Kuo, A. Gurjar, P. Biswas, and A. Shrivastava, "DSP-MLIR: A domain-specific language and MLIR dialect for digital signal processing," in *Proceedings of the 26th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, Tools and Theory of Embedded Systems (LCTES)*, 2025.
- [5] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O'Reilly, and S. Amarasinghe, "Opentuner: An extensible framework for program autotuning," in *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2014. [Online]. Available: <https://dl.acm.org/doi/10.1145/2628071.2628092>
- [6] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proceedings the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/chen>
- [7] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Anso: Generating high-performance tensor programs for deep learning," in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [8] R. T. Mullapudi, A. Adams, D. Sharlet, J. Ragan-Kelley, and K. Fatahalian, "Automatically scheduling halide image processing pipelines," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 35, no. 4, 2016. [Online]. Available: <https://dl.acm.org/doi/10.1145/2897824.2925952>
- [9] F. Agakov, E. Bonilla, J. Cavazos, B. Franke, G. Fursin, M. O'Boyle, J. Thomson, and C. Williams, "Using machine learning to focus iterative optimization," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2006. [Online]. Available: <https://dl.acm.org/doi/10.1109/CGO.2006.37>

- [10] S. Kulkarni and J. Cavazos, "Mitigating the compiler optimization phase-ordering problem using machine learning," in *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, 2012. [Online]. Available: <https://dl.acm.org/doi/10.1145/2384616.2384628>
- [11] H.-I. C. Liu, M. Brehler, M. Ravishankar, N. Vasilache, B. Vanik, and S. Laurenzo, "Tinyiree: An ml execution environment for embedded systems from compilation to deployment," *arXiv preprint arXiv:2205.14479*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14479>
- [12] IREE Project, "Iree codegen dialect: Dispatchloweringpasspipelineattr," <https://iree.dev/reference/mlir-dialects/IREECodegen/>, accessed 2026-03-07.
- [13] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: A compiler infrastructure for the end of moore's law," *arXiv preprint arXiv:2002.11054*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.11054>
- [14] IREE Project, "Matrix multiplication with mmt4d," <https://iree.dev/community/blog/2021-10-13-matrix-multiplication-with-mmt4d/>, accessed 2026-03-07.
- [15] —, "Turn the data-tiling path on by default for cpu backends (iree discuss thread)," <https://groups.google.com/g/iree-discuss/c/lpQzYXw1r6s>, accessed 2026-03-07.
- [16] Z. Jia, O. Padon, J. Thomas, T. Todd, A. Aiken, and M. Zaharia, "TASO: optimizing deep learning computation with automated generation of graph substitutions," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 47–62.
- [17] J. Shao, X. Zhou, S. Feng, B. Hou, R. Lai, H. Jin, W. Lin, M. Masuda, C. H. Yu, and T. Chen, "Tensor program optimization with probabilistic programs," *Advances in Neural Information Processing Systems*, vol. 35, pp. 35 783–35 796, 2022.
- [18] A. Adams, K. Ma, L. Anderson, R. Baghdadi, T.-M. Li, M. Gharbi, B. Steiner, S. Johnson, K. Fatahalian, F. Durand, and J. Ragan-Kelley, "Learning to optimize halide with tree search and random programs," *arXiv preprint arXiv:1911.08152*, 2019. [Online]. Available: https://halide-lang.org/papers/halide_autoscheduler_2019.pdf
- [19] A. Brauckmann, A. Goens, and J. Castrillon, "Polygym: Polyhedral optimizations as an environment for reinforcement learning," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021, pp. 17–29.
- [20] D. R. Lamouri, I. N. Aouadj, S. Kourta, and R. Baghdadi, "Pearl: Automatic code optimization using deep reinforcement learning," in *Proceedings of the 39th ACM International Conference on Supercomputing*, 2025, pp. 959–974.
- [21] M. Tirichine, N. Ameur, N. Bendib, I. N. Aouadj, B. Djad, R. Bouloudene, and R. Baghdadi, "A reinforcement learning environment for automatic code optimization in the mlir compiler," *arXiv preprint arXiv:2409.11068*, 2024.
- [22] A. Haj-Ali, Q. J. Huang, J. Xiang, W. Moses, K. Asanovic, J. Wawrzyniec, and I. Stoica, "Autophase: Juggling hls phase orderings in random forests with deep reinforcement learning," *Proceedings of machine learning and systems*, vol. 2, pp. 70–81, 2020.
- [23] Y. Liang, K. Stone, A. Shamel, C. Cummins, M. Elhoushi, J. Guo, B. Steiner, Y. Yang, Y. Tian, and H. Leather, "Learning compiler pass orders using coresets and normalized value prediction," *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [24] C. Deng, J. Wu, N. Feng, J. Wang, and M. Long, "Compilerdream: Learning a compiler world model for general code optimization," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 486–497.
- [25] M. Ganai, H. Li, T. Enns, Y. Wang, and R. Huang, "Target-independent xla optimization using reinforcement learning," *arXiv preprint arXiv:2308.14364*, 2023.
- [26] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, "Mlgo: A machine learning guided compiler optimizations framework," *arXiv preprint arXiv:2101.04808*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.04808>
- [27] A. Haj-Ali, N. K. Ahmed, T. Willke, S. Shao, K. Asanovic, and I. Stoica, "Neurovectorizer: End-to-end vectorization with deep reinforcement learning," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2020. [Online]. Available: <https://dl.acm.org/doi/10.1145/3368826.3377928>
- [28] C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner, Y. Tian, and H. Leather, "Compilergym: Robust, performant compiler optimization environments for ai research," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, 2022. [Online]. Available: <https://arxiv.org/abs/2109.08267>
- [29] M. Jantz and P. Kulkarni, "Analyzing and addressing false interactions during compiler optimization phase ordering," *Software: Practice and Experience*, vol. 44, 06 2014.
- [30] P. Kulkarni, D. Whalley, G. Tyson, and J. Davidson, "Exhaustive optimization phase order space exploration," in *International Symposium on Code Generation and Optimization (CGO'06)*, 2006, pp. 13 pp.–318.
- [31] S. Triantafyllis, M. Vachharajani, N. Vachharajani, and D. I. August, "Compiler optimization-space exploration," in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*, ser. CGO '03. USA: IEEE Computer Society, 2003, p. 204–215.
- [32] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017.
- [33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [34] S. Huang and S. Ontañón, "A closer look at invalid action masking as a policy gradient," in *2022 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 16–22.
- [35] T. Chen, L. Zheng, E. Yan, Z. Jiang, T. Moreau, L. Ceze, C. Guestrin, and A. Krishnamurthy, "Learning to optimize tensor programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. [Online]. Available: <https://papers.nips.cc/paper/2018/hash/8b5700012be65c9da25f49408d959ca0-Abstract.html>



Megan Kuo Megan Kuo received the B.B.A. degree in data science and quantitative finance from National Tsing Hua University, Hsinchu, Taiwan, in 2023, and the M.S. degree in computer science from Arizona State University, Tempe, AZ, USA, in 2026. She is currently an AI Compiler Software Engineer at Samsung Semiconductor, San Jose, CA, USA. Her research interests include compilers, machine-learning compilers, and multi-level intermediate representations.



Atharva Khedkar Atharva Khedkar (Graduate Student Member, IEEE) received the B.Tech. degree in artificial intelligence from G. H. Raisoni College of Engineering, Nagpur, India, in 2023. He is currently pursuing the Ph.D. degree in computer science with Arizona State University, Tempe, AZ, USA. His research interests include compilers, machine-learning accelerators, and multi-level intermediate representation.



Aviral Shrivastava Aviral Shrivastava (Senior Member, IEEE) received the B.Tech. degree in computer science and engineering from the Indian Institute of Technology Delhi, New Delhi, India, in 1999, and the M.S. and Ph.D. degrees in information and computer science from the University of California, Irvine, CA, USA, in 2002 and 2006, respectively. He is currently a Professor with the School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ, USA, where he directs the Make Programming Simple Lab. His research interests include compilers, computer architecture, and machine-learning acceleration. He is the Editor-in-Chief of IEEE Embedded Systems Letters and a recipient of the NSF CAREER Award. He was an Associate Editor of this TRANSACTIONS from 2018 to 2023.